

소켓프로그래밍3

김학재

2026.08.11

목차

1. TCP 프로그래밍 흐름
2. TCP 서버 구현 방법 (1) fork
3. TCP 서버 구현 방법 (2) thread
4. TCP 서버 구현 방법 (3) select
5. epoll 방식

지난 이야기

1.socket()

2.주소설정

3.bind()

4.listen()

5.accept()

6.write() / read()

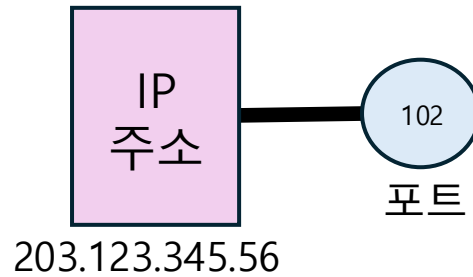
7.close()

1.Socket()

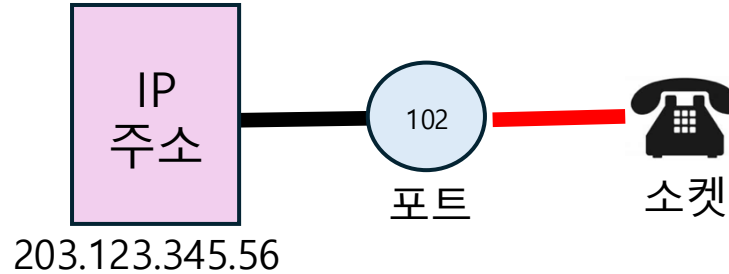


소켓

2.주소설정



3.bind()
4.listen()

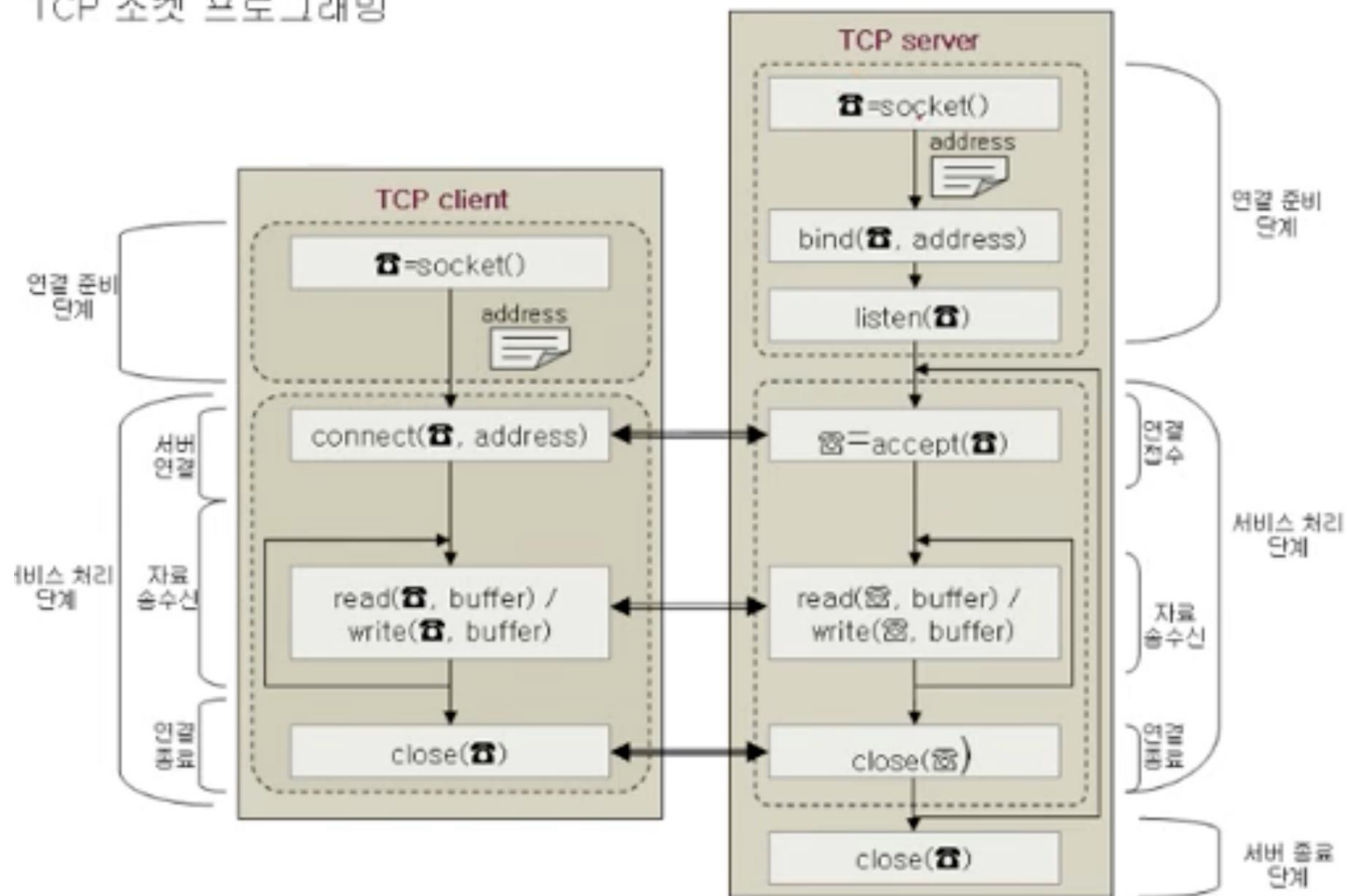


5.accept()

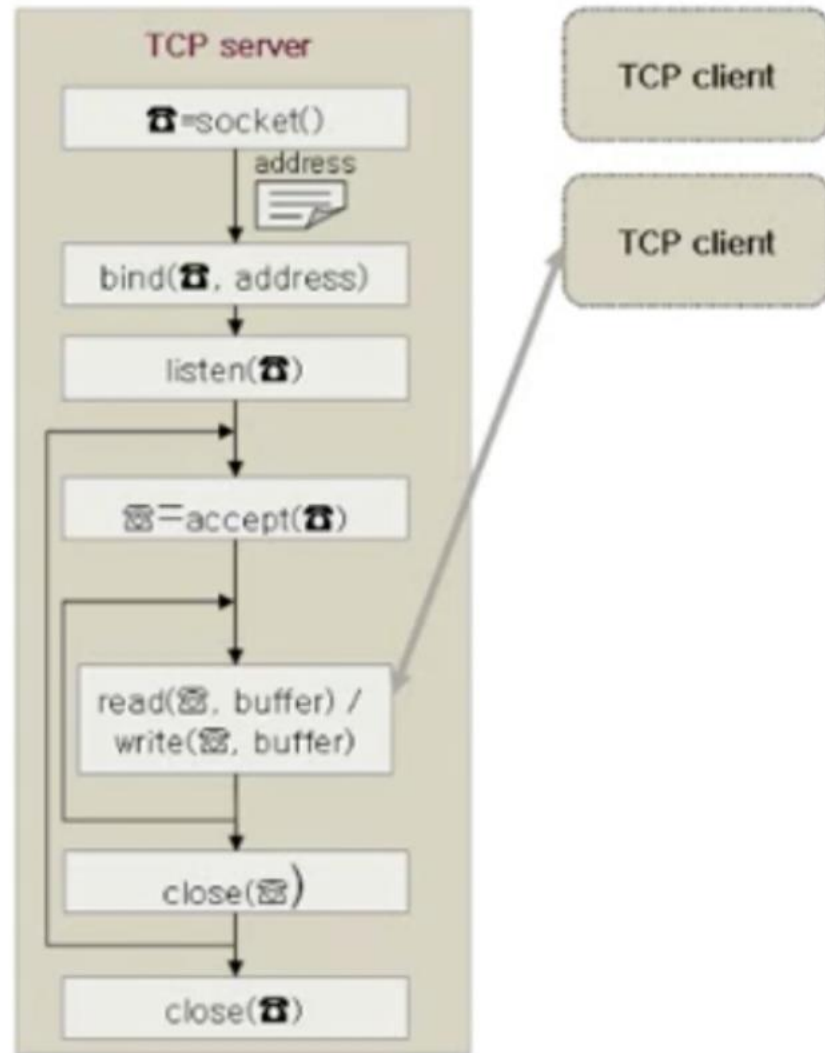


클라이언트 - 서버 개요도

TCP 소켓 프로그래밍



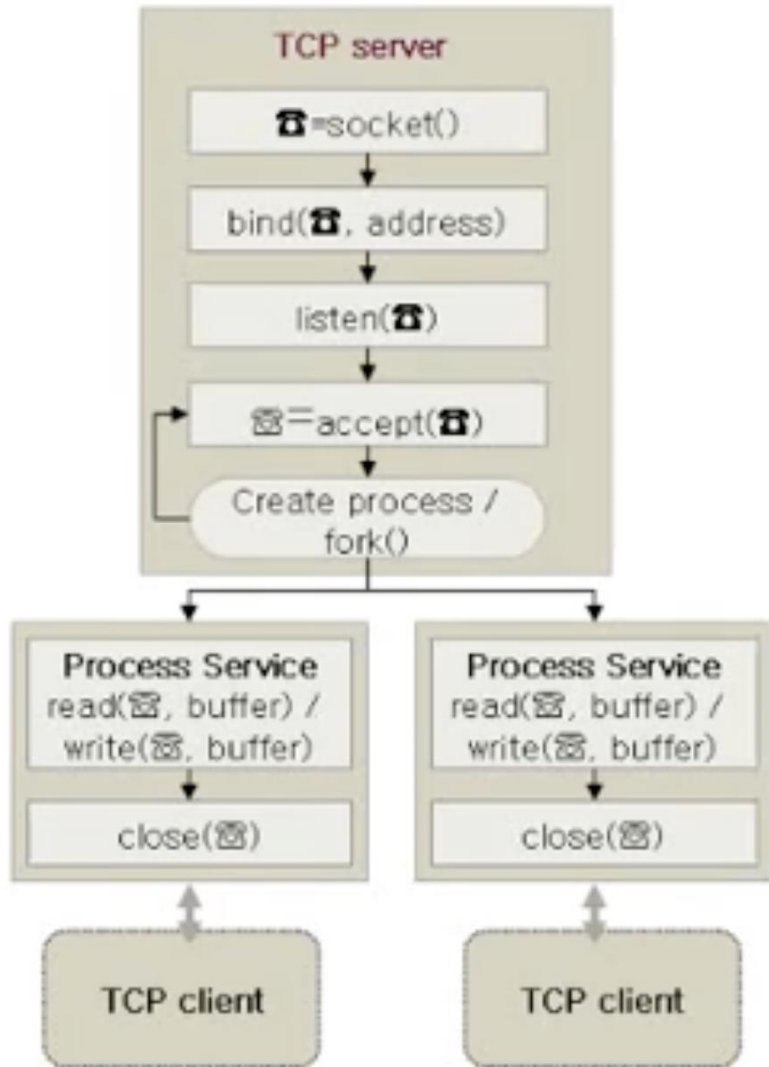
TCP 서버 구현 방법 - 단일접속 서버



단일서버

1. 한 번에 한 클라이언트만 처리하는 구조
2. 서버 프로세스 하나가 모든 클라이언트를 처리한다.

TCP 서버 구현 방법 (1) - fork



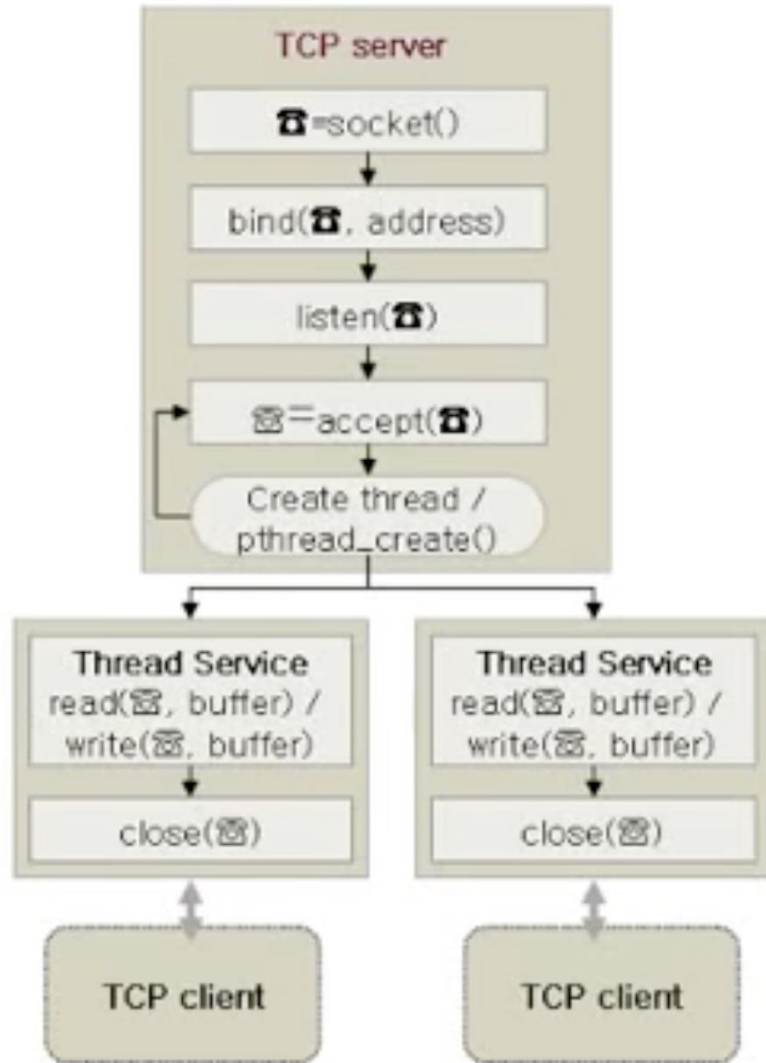
```
1  while (1) {
2      conn_s = accept(listen_s, ...);
3
4      if ((pid = fork()) > 0) { // 부모 프로세스
5          // 다른 클라이언트의 요청 접수
6          close(conn_s);
7          continue;
8      }
9      else if (pid == 0) { // 자식 프로세스
10         // 클라이언트의 요청 처리
11         close(listen_s);
12         do_service(conn_s);
13         exit(0);
14     }
15 }
```

부모 :
계속 받는 역할
Listen 소켓 역할

자식 :
계속 통신 역할
연결 소켓 역할

부모는 계속 새로운 클라이언트 받아서 자식에게 일을 맡기는 역할,
자식은 실제 통신을 담당하는 역할
단일접속 서버와 다르게 한번에 여러 클라이언트와 통신 가능

TCP 서버 구현 방법 (2) – thread



```
1 while (1) {
2     conn_s = accept(listen_s, ...);
3
4     pthread_create( ... );
5 }
```

Listen 소켓은 하나만 존재

Accept()할 때마다 새로운 연결용 소켓을 계속 만들어냄.

멀티스레딩은 메모리를 공유한다는게 특징이다

이것을 통해 통신하면 데이터 공유가 편하다는 장점이 있지만,

하지만 공유 때문에 경쟁상태가 되면서 문제가 생기기도 한다.

Fork vs Thread 방식

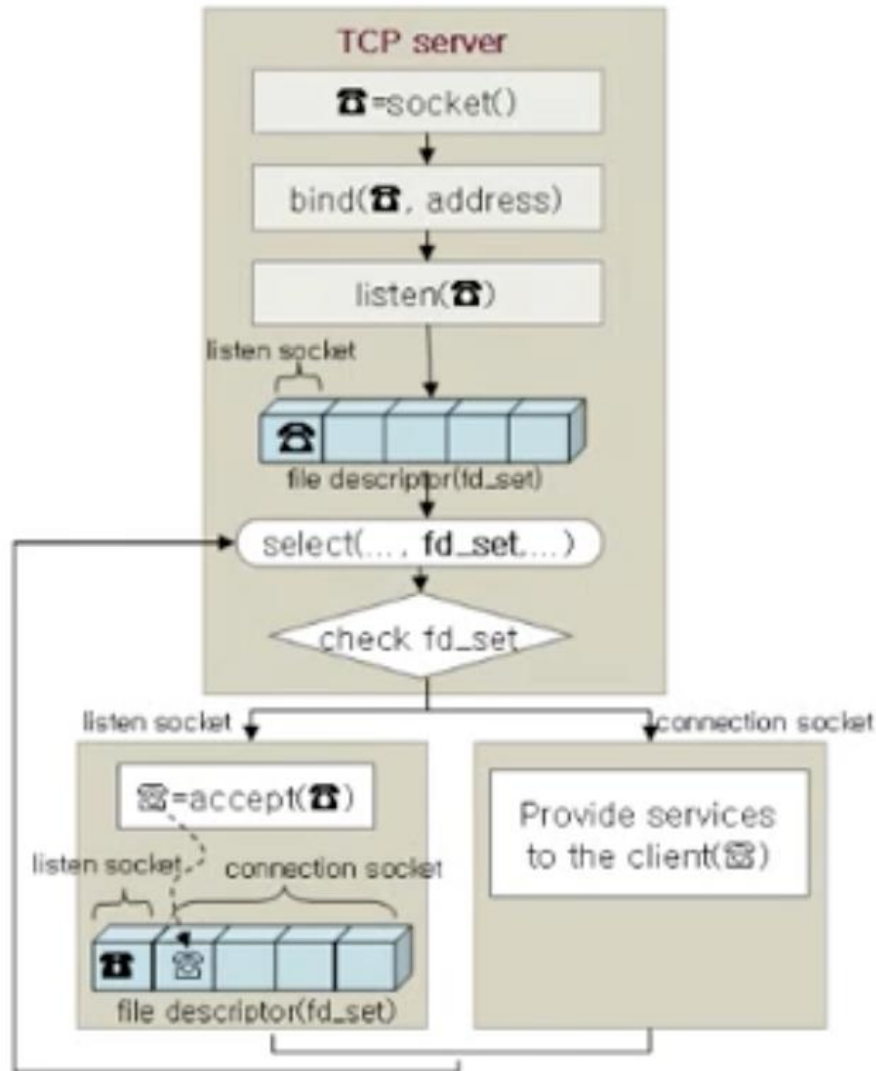
	Fork (멀티프로세싱)	Thread (멀티스레딩)
크기	무거운 프로세스	가벼운 스레드
메모리공유	Fork생성시 메모리 완전분리	Code, Data, Heap 모든 스레드가 공유
생성속도	프로세스 통째로 복사하므로 cpu/메모리 오버헤드가 큼	생성속도 빠름
안정성	자식프로세스 하나 죽어도 다른 프로세스에 영향 없음	스레드 하나가 치명적 오류내면, 서버전체가 죽음
데이터통신	독립된 메모리라 IPC(파이프, 메모리)	동기화, Lock 중요

결론 :

Thread 방식은 가벼워서 **대규모 트래픽을 처리하는 서버의 표준**처럼 사용, 하지만 메모리를 공유하기 때문에 Mutex등을 활용한 동기화 처리 신경써야함.

Fork 방식은 구조가 직관적이고 안정성이 매우 뛰어나. 하지만 대규모 클라이언트가 있다면 서버 메모리 오버헤드 너무 큼.

TCP 서버 구현 방법 (3) – select



멀티플렉싱 방법

스레드나 프로세스를 여러 개 만들지 않고

단, 1개의 메인 스레드(프로세스)만으로 여러 명의 클라이언트를 동시에 처리한다.

`fd_set`이라는 배열존재 (소켓 번호 저장) (파일디스크립터)

`select`함수가 이 배열을 처음부터 쪽 훑으면서 검사한다.

-어떤 소켓에서 메시지를 보냈는지,

-리슨 소켓에 반응이 왔는지 (새 클라이언트가 왔는지)

한번 처리 후 blocking 모드로 들어감.

그리고 배열에 있는 소켓 중 하나라도 반응이 오면 이제 다시 작동.

TCP 서버 구현 방법 (3) – select

```
1  while(1) {
2      FD_SET(listen_s, &fd_set);
3      FD_SET(연결소켓목록, &fd_set);
4      select(..., fd_set, ...);
5
6      if(FD_ISSET(listen_s, fd_set)){
7          connSock = accept(listen_s);
8          add_fd_set(connSock);
9      } 리슨소켓 반응
10         -> 연결소켓 추가
11
12      for(s ∈ connection sockets) {
13          if(FD_ISSET(s)) {
14              do_client_service(s);
15          }
16      } 연결소켓 반응
17         -> 통신
```

fd_set 배열에 listen_s 번호 추가. (리슨소켓)

fd_set 배열에 연결소켓들 번호 추가.

bit 0 -> 1로 바꾸는것임

select() : fd_set에 있는것들 감시

어차피 리슨소켓은 계속 켜놓을것이고
번호가 계속 고정일텐데 왜 반복할때마다
fd_set에 번호를 추가하는지에 대하여.

-> select 특성상 한번 fd_set을 싹 훑어볼 때
이벤트가 발생한 소켓의 비트만 1로 냅두고
아무일도 없었던 소켓들의 비트를 0으로 바
꿔버리기 때문이다.

Thread vs Select 방식

결국 thread 방식도 fork에 비해선 가볍지만 스레드를 많이 만들다보면 무거워지기 마련 경쟁상태(race condition)발생한다는 단점

그에 반면 select 방식은 스레드 하나로 통신.

그래도 멀티스레드방식이 빠른거아니냐? 라고한다면 select에 대한 장점은 cpu를 많이 사용하지않고 네트워크 I/O를 많이 사용하는 상황에서 만약 클라이언트가 10000명이 연결되어있다고 칠 때

실제 데이터를 보내는 사람이 5명이라고 할 때

Thread 방식은 대부분의 스레드가 데이터 기다리는중.

select 방식은 fd_set을 확인하고 지금 데이터 온 건 5개에 대하여만 처리한다는것이다.

여러 방식

thread방식에서 발전 -> Thread Pool (스레드 풀)

스레드를 클라이언트 올때마다 만드는게 아닌 미리 많이 만들어 두는 것
기존의 스레드 방식은 생성하고 통신 끝나면 삭제한다.

하지만 스레드 풀 방식도 컨텍스트 스위칭 너무 많이하면 느려짐

select 방식의 단점을 고친 epoll 방식 (현재 최고의 방식)

운영체제(커널) 안에 epoll 인스턴스를 하나 만든다.

소켓을 추가할 때 한 번만 운영체제에 알려주면 OS가 알아서 감시한다.

(select처럼 매번 fd_set배열을 덮어쓰고 해줄 필요가없다)

감사합니다